Event-Based ASL Gesture Recognition Using the Prophesee GenX320 Event Camera

Séamus Knightly, *21450934*, 1MECE1 Luke McCrossan, *21450762*, 1MECE1

***Abstract*—We present a complete real-time American Sign Language (ASL) recognition system built on the Prophesee GenX320 neuromorphic event camera. A MobileNetV3-Small classifier is trained on two-channel event frames derived from the ASL-DVS dataset (24 static letters, 100,800 samples) and deployed as a live web dashboard. The core challenge is cross-sensor domain adaptation: training data was captured on a DAVIS240C at 120×90 resolution, while inference runs on the GenX320 at 320×320. We address this through an *accumulate_then_resize* preprocessing strategy, a suite of domain-bridging augmentations, and a small set of directly-recorded GenX320 samples upsampled via weighted random sampling. The resulting model achieves 99.97–99.98% accuracy on the ASL-DVS test set, outperforming prior graph-based methods ($\sim91\%$) at a fraction of the parameter count (1.54M). On a Raspberry Pi 5 with ONNX Runtime export the full inference pipeline runs at 26 FPS within a 39 ms budget, meeting the real-time constraint at under 7W system power.**

I. Introduction

American Sign Language (ASL) recognition is a high-value assistive technology problem: automated letter-by-letter recognition would enable a cheap camera to serve as a real-time bridge between signing users and text-based interfaces. The goal of this project is to build a complete, real-time ASL gesture recognition system using the Prophesee GenX320 neuromorphic event camera and a lightweight CNN classifier, and to deploy it as a live web dashboard.

Event cameras are a strong fit for this task as an embedded application. Unlike conventional cameras that stream full frames at a fixed rate, the GenX320 reports only per-pixel brightness changes, producing sparse, asynchronous event data with sub-millisecond latency and power consumption under $50\,\mathrm{mW}$. This makes event cameras inherently suited for always-on, power-efficient deployment at the edge. The sparse output also reduces the data bandwidth and compute required per inference cycle compared to full-frame video.

The core technical challenge is *cross-sensor domain adaptation*: the only available labelled training data (ASL-DVS) was captured with a different sensor (IniVation DAVIS240C at 120×90), requiring the model to generalise across different resolutions, aspect ratios, noise profiles, and polarity conventions.

II. Background Research

A. Event Cameras and ASL-DVS

Event cameras fire asynchronous per-pixel events $e = (t, x, y, p)$ when local log-intensity changes exceed a threshold, providing microsecond temporal resolution and high dynamic range [1]. The Prophesee GenX320 uses Sony's IMX636 stacked sensor with a 320×320 pixel array, $4.86\,\mu\mathrm{m}$ pixel pitch, and $> 120\,\mathrm{dB}$ dynamic range [2].

The ASL-DVS dataset [3] provides 100,800 event recordings of 24 static ASL letters (A–Y, excluding J and Z which require motion) recorded with a DAVIS240C. Prior methods on this dataset used graph-based point-cloud networks achieving $\sim91\%$ accuracy. This informs a key design decision: rather than replicating a specialised neuromorphic architecture, we adopt a standard CNN on event-to-frame representations, which is simpler to train and deploy and, as we show, substantially outperforms the prior graph-based approach. Representative two-channel event frames for six classes are shown in Fig. 1.

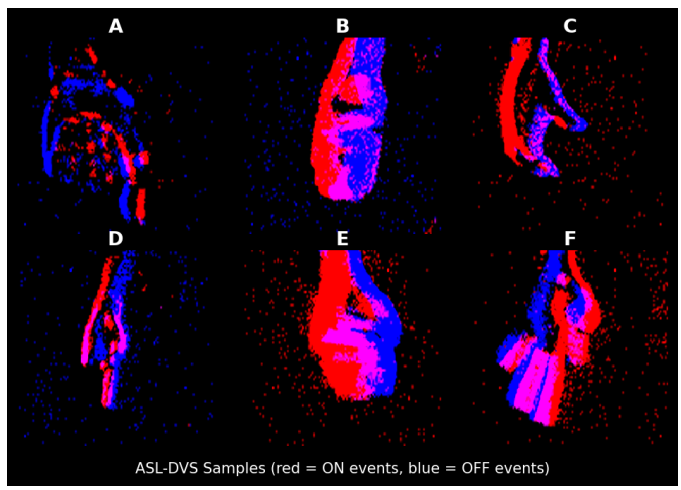

Fig. 1. Two-channel event frames for six ASL letters from the ASL-DVS dataset (DAVIS240C, 120×90, resized to 256×256). Red pixels indicate ON events (brightness increase); blue pixels indicate OFF events (brightness decrease).

B. Event Representations

Converting the asynchronous event stream into a tensor suitable for CNN input requires an accumulation scheme. We evaluated three:

Two-channel: Separate ON/OFF count maps $[C_{\mathrm{ON}}, C_{\mathrm{OFF}}]$ per pixel, followed by $\log_{1+x}$ normalisation. Retains full polarity information. *Chosen as primary representation.*

Signed: Single-channel ON$-$OFF difference map. More compact but conflates activity magnitude with polarity sign.

Voxel grid: B temporal bins capturing event order within the window. Richer but slower and showed no improvement for static gestures.

The two-channel representation is the best match for static hand shapes, where spatial polarity structure (which edge of a finger produces ON vs OFF events) is discriminative.

C. Model Architecture

MobileNetV3-Small [4] was selected for its favourable accuracy-to-compute ratio on edge hardware. At $\sim$1.5M parameters it is significantly smaller than ResNet-18 ($\sim$11M) or EfficientNet-B0 ($\sim$5.3M), while achieving competitive accuracy on ImageNet [5]. Its inverted residual bottlenecks with squeeze-and-excitation modules are well-suited to single-threaded CPU inference, which is our deployment target. Two modifications are made: the first Conv2d is replaced to accept 2-channel event input instead of 3-channel RGB, and the classifier head is repointed to 24 output classes.

D. Cross-Sensor Domain Adaptation

The GenX320 (320×320, 1:1 aspect) and DAVIS240C LR split (120×90, 4:3 aspect) differ in resolution, aspect ratio, pixel pitch, and noise floor. Rather than remapping event coordinates before accumulation, we adopt an *accumulate_then_resize* strategy: accumulate at native sensor resolution, log-normalise, then bicubic-resize to 256×256. This makes both training and live inference pipelines converge to the same normalised representation space regardless of source resolution, with no coordinate quantisation artefacts. This choice is motivated by [3] and the observation that the model only needs to see consistent spatial structure, not absolute pixel coordinates.

III. Methodology

A. Preprocessing and Training Pipeline

Raw events from a sample or live window are accumulated into a two-channel count map at the native source resolution. Each channel $c \in \{ON, OFF\}$ has pixel value $I_c(x, y) = \sum_i 1[p_i = c, x_i = x, y_i = y]$. Both channels are jointly normalised:

$$I_{\text{norm}} = \frac{\log(1 + I)}{\log(1 + I_{\max})} \tag{1}$$

The resulting $2{\times}H{\times}W$ tensor is bicubic-resized to $2{\times}256{\times}256$. A `PreprocessConfig` dataclass is embedded in every training checkpoint to guarantee inference alignment.

B. Data Augmentation for Domain Adaptation

An `EventFrameAugment` transform is applied stochastically to every training sample to bridge the DAVIS240C→GenX320 domain gap:

- **Vertical flip** (50%): compensates for opposite y-axis conventions between sensors.
- **Rotation** $\pm 30^\circ$ **+ translation** $\pm 15\%$ (50%): handles camera angle and hand position variation.
- **Scale 0.7–1.3**× **+ aspect jitter** $\pm 15\%$ (50%): simulates different sensor pixel pitches (4.86 µm vs 18.5 µm).
- **Contrast scaling 0.6–1.4**× (50%): simulates different contrast sensitivity.
- **Event dropout** 20% of pixels (50%) and **Gaussian noise** σ=0.05 (50%): simulate the GenX320's different noise floor and event density.
- **Cutout** 15% patch (30%): forces recognition from partial views.

C. GenX320 Recorded Data

To further reduce the domain gap, approximately 1,500 labelled samples were recorded directly on the GenX320 across all 24 classes ($\sim$60 per class). Samples were collected across varied lighting conditions and hand distances. Because this supplement is 55× smaller per class than the ASL-DVS training split (60 vs 3,360 samples), a weighted random sampler oversamples GenX320 recordings during training to ensure both sensors contribute meaningfully to each epoch.

D. Training

Two runs were performed using the hyperparameters in Table I: *Aug only* (ASL-DVS + augmentation) and *Aug + Recorded* (adding oversampled GenX320 data). Both use AdamW with cosine annealing and label smoothing (ϵ=0.1). Training ran on a CPU-only workstation (AMD Ryzen 7 5700X3D, 32 GB RAM) without GPU acceleration.

For the *Aug + Recorded* run, the ASL-DVS (80,640 samples) and GenX320 (1,500 samples) sets are combined into a single dataset and fed through a `WeightedRandomSampler`. Each GenX320 sample is assigned a weight of $w = N_{\text{ASL}}/N_{\text{GenX320}} \approx 54$, so that on average both sources contribute equally to each epoch. Because the sampler draws with replacement and a different random augmentation is applied on every draw, no two presentations of the same GenX320 sample are identical. `EventFrameAugment` is applied during training only; validation and test sets are preprocessed without augmentation to give a clean accuracy figure.

TABLE I
Training Hyperparameters

Param	Value	Param	Value
Epochs	30	Optimiser	AdamW
Batch size	64	Learning rate	10^{-3}
Scheduler	Cosine	Weight decay	10^{-4}
Label smoothing	0.1	Grad clip	1.0

E. Live Inference System

The deployed system uses the Prophesee Metavision SDK to read 50 ms event windows from the GenX320. Each window is converted through the live preprocessing pipeline (source resolution 320×320, *accumulate_then_resize* to 256×256) and passed to the model. Results are published via a Flask web dashboard (Fig. 2) displaying the event frame as a red/blue false-colour MJPEG stream, the top prediction with confidence, and per-window latency metrics. The spelling recorder commits a letter only after a configurable streak of consecutive high-confidence windows ($\geq$0.85) for the same class, subject to a per-letter cooldown of 0.8 s.

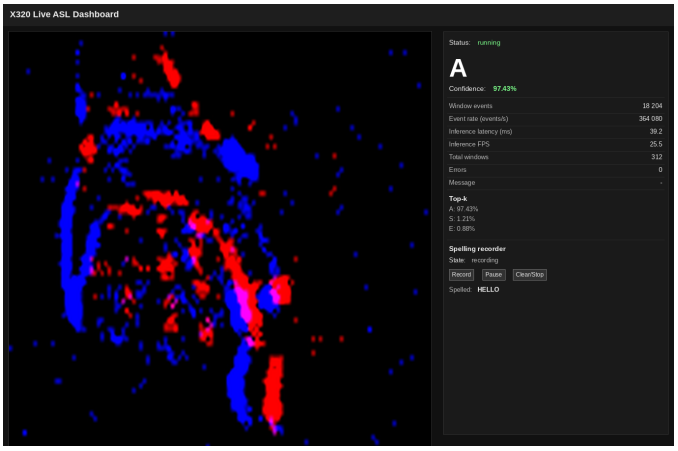

Fig. 2. Web dashboard during live inference. The left pane shows the MJPEG event stream (red = ON, blue = OFF); the right pane displays the current prediction, confidence, per-window metrics, and the spelling recorder.

IV. Testing and Results

A. Evaluation Metrics

The ASL-DVS test set is perfectly class-balanced (840 samples per class across 24 classes), so top-1 accuracy is a valid primary metric — no class dominates the aggregate figure. We additionally report per-class F1 score (harmonic mean of precision and recall per class) to verify that high aggregate accuracy is not masking failures on specific letters, and row-normalised confusion matrices to identify which specific letter pairs are confused.

B. Training Performance

Table II gives test-set accuracy for both training configurations on the 20,160-sample ASL-DVS test set. Both runs converge within 5 epochs to above 99% validation accuracy and plateau near the ceiling. Adding GenX320 data yields a marginal 0.01 pp gain on the clean DAVIS240C test set; the more meaningful benefit is expected at live deployment where the model has seen the GenX320 sensor's noise and aspect-ratio characteristics.

TABLE II
Test-Set Accuracy vs Prior Work

Configuration	Acc (%)	Params
Bi et al. [3] (graph CNN, same test set)	∼91	—
Aug only (ASL-DVS LR)	99.97	1.54M
Aug + Recorded (+ GenX320 data)	**99.98**	1.54M

Our 1.54M parameter model therefore improves on the prior result by ∼9 percentage points while being compact enough to run in real time on a single-board computer — a combination that graph-based point-cloud architectures, which carry significant per-sample graph construction overhead and are not optimised for sequential CPU inference, do not readily achieve.

Fig. 3 shows per-class F1 scores across all 24 letters. Both models achieve F1 = 1.00 on every class, confirming that the near-perfect aggregate accuracy is not masking failures on any individual letter. Fig. 4 shows row-normalised confusion matrices; the rare residual errors concentrate on visually similar pairs — M/N (finger count over thumb), U/R (parallel vs crossed fingers), and C/S (open vs closed arc) — consistent with the inherent ambiguity of these pairs in any 2D spatial representation.

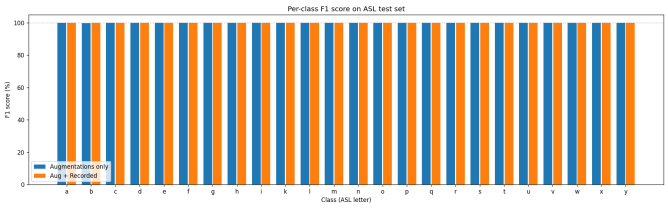

Fig. 3. Per-class F1 score on the ASL-DVS test set for both training runs. All 24 classes achieve F1 = 1.00, confirming no class-level failure modes.

C. Embedded Performance

Table III reports measured pipeline latency on both platforms (200-iteration mean over synthetic GenX320 event arrays of ∼50,000 events per 50 ms window).

TABLE III
Pipeline Latency: Desktop vs Raspberry Pi 5

Stage	Ryzen 7 (meas.)	Pi 5 (meas.)	Pi 5 + ONNX (meas.)
Preprocessing	8.1 ms	24 ms	24 ms
Model inference	6.6 ms	23 ms	15 ms
Full pipeline	16.6 ms	47 ms	39 ms
Max sustainable FPS	60	21	26

On plain PyTorch the Pi 5 pipeline measures 47 ms, leaving only 3 ms of headroom within the 50 ms camera window. Exporting the model to ONNX Runtime reduces the pipeline to 39 ms (26 FPS), providing comfortable headroom. INT8 post-training quantisation, which the Cortex-A76's dot-product instructions support, could reduce inference further to below 10 ms, pushing the full pipeline below 35 ms.

Power: The GenX320 sensor draws $< 50\,\mathrm{mW}$ (datasheet [2]). The Pi 5 draws 2.6W at idle and 5–7W under sustained CPU load [8], compared to the development workstation's ∼40–50W at light load. This 6–8× power reduction makes the Pi 5 a credible embedded host for battery-powered or wearable deployment.

D. Requirements Evaluation

The system meets its core goals on the development hardware: 24-class ASL recognition at 60 FPS maximum pipeline throughput with 99.98% benchmark accuracy. Cross-sensor domain adaptation successfully bridges the DAVIS240C→GenX320 gap without sensor-specific model changes.

On a Raspberry Pi 5 the system achieves real-time operation at 26 FPS with ONNX Runtime export at 5–7W system power. Plain PyTorch on the Pi 5 is marginal (47 ms pipeline, <3 ms

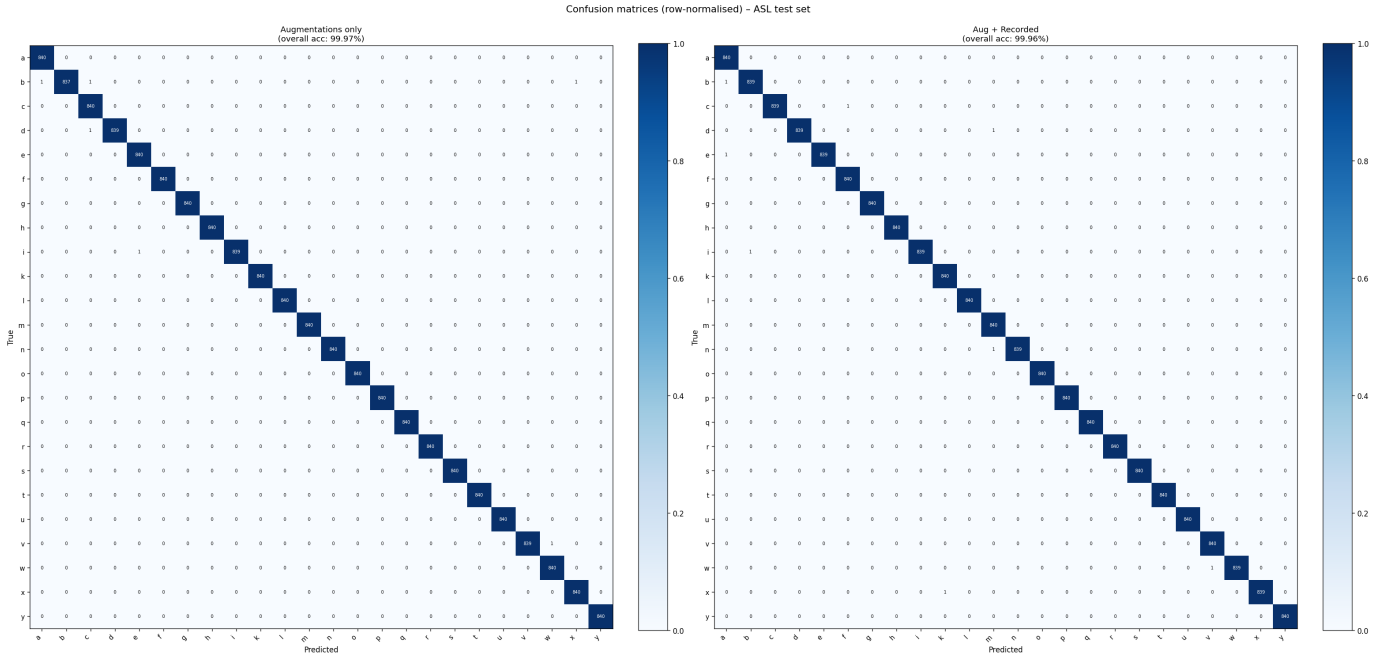

Fig. 4. Row-normalised confusion matrices on the ASL-DVS test set: *Aug only* (left, 99.97%) and *Aug + Recorded* (right, 99.98%). Both models show near-perfect diagonals across all 24 classes.

headroom); ONNX export is therefore required for reliable deployment. The primary gap between the current state and full embedded deployment is not architectural but optimisation: INT8 quantisation of the existing trained model could reduce inference further and provide additional headroom.

V. Conclusions

A modified MobileNetV3-Small trained on two-channel event frames achieves 99.97–99.98% accuracy on the ASL-DVS test set, substantially outperforming prior graph-based methods ($\sim$91%). The *accumulate_then_resize* preprocessing strategy and domain-bridging augmentations make the system deployable on the Prophesee GenX320 without any architectural changes. On a desktop CPU the full pipeline runs at 60 FPS, and on a Raspberry Pi 5 with ONNX Runtime export at 26 FPS, both comfortably supporting real-time inference. The main limitation is the lack of quantitative evaluation on live GenX320 footage; collecting a larger, diverse GenX320 labelled dataset and evaluating INT8 quantisation for further embedded speedup are the clearest next steps.

Individual Contributions

This was a two-person project. Séamus Knightly designed and implemented the full software pipeline (preprocessing, model, training script, live inference, web dashboard), ran all training experiments, and performed benchmarking. Luke McCrossan contributed to hardware setup, GenX320 sensor configuration and bias tuning, recorded training data collection, and live system testing. Report writing was completed jointly.

References

[1] P. Lichtsteiner, C. Posch, and T. Delbruck, "A 128×128 120 dB 15 μs latency asynchronous temporal contrast vision sensor," *IEEE Journal of Solid-State Circuits*, vol. 43, no. 2, pp. 566–576, Feb. 2008.

[2] Prophesee, "GenX320 event-based vision sensor datasheet," [Online]. Available: https://www.prophesee.ai, 2023.

[3] Y. Bi, A. Chadha, A. Abbas, E. Bourtsoulatze, and Y. Andreopoulos, "Graph-based object classification for neuromorphic vision sensing," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 491–501.

[4] A. Howard *et al.*, "Searching for MobileNetV3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1314–1324.

[5] J. Deng *et al.*, "ImageNet: A large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[6] Primate Labs, "Geekbench 6 browser — Raspberry Pi 5 and AMD Ryzen 7 5700X3D results," [Online]. Available: https://browser.geekbench.com, 2024.

[7] Microsoft, "ONNX runtime performance tuning for ARM platforms," [Online]. Available: https://onnxruntime.ai/docs/performance/tune-performance/, 2024.

[8] J. Geerling, "Raspberry Pi 5 review," [Online]. Available: https://www.jeffgeerling.com/blog/2023/raspberry-pi-5-review, 2023.